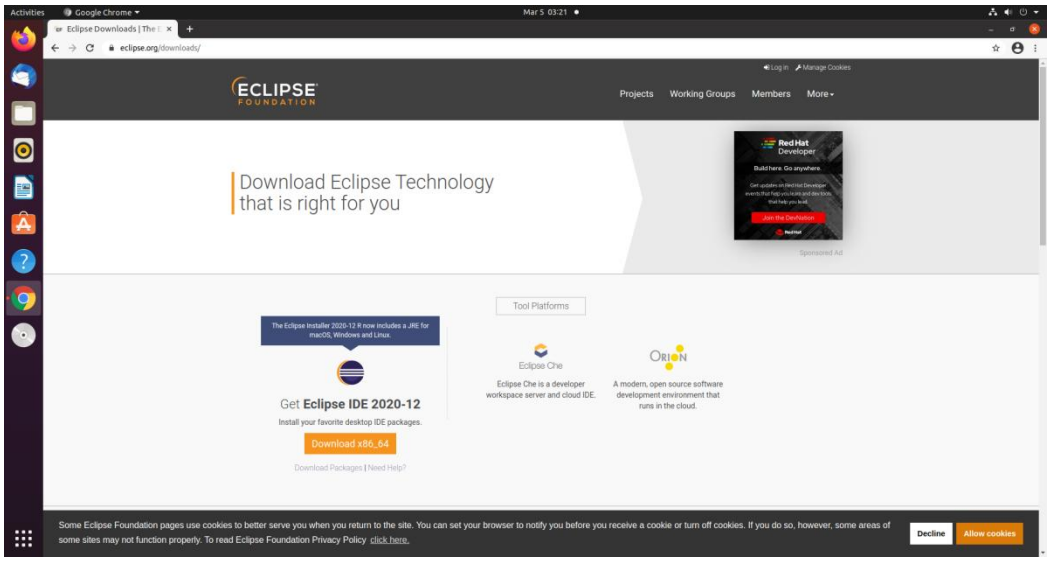
Brian Simpson
OS 4600

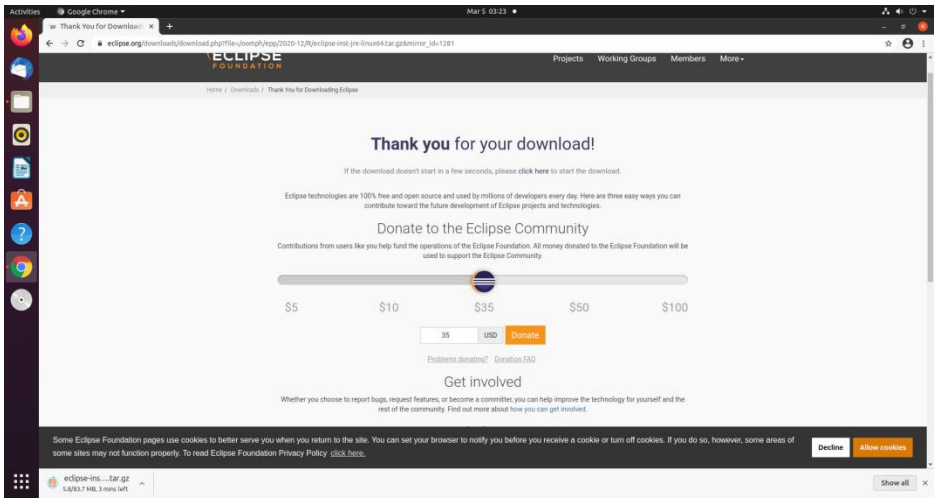
Days Worked:

- 03/05: Set up the coding environment (Eclipse IDE), and downloaded Santa.zip to begin working on understanding the different components of the Santa scenario and how they interact with each other (Step 0).
    - Time taken: ~2 hours
- 03/06: Fixed the layout of my directories of the Eclipse Workspace to accurately represent the desired format of professor Boloni for the submission (step0, step1 subdirectories etc.). I also began construction of this journal at this point.
    - Time taken: 30 minutes
- 03/06: Solved step 1 by causing all of the active threads to stop through the usage of deferred termination once the simulation reaches day 370.
    - Time taken: ~3 hours (had to do a lot of individual research on deferred termination)
- 03/09: Worked on step 2 for about 5 or 6 hours without making much progress, ran into a lot of Java roadblocks as I kept trying to code Java as if it were C which was a mistake.
- 03/11: Worked on step 2 for another 2 or 3 hours, finally solving it via the use of a listIterator through the arrayList of elves in order to check their states.
- 03/11: Solved step 3 in about 2 hours of work, wasn’t too bad at all since it was just a harder version of the concept I finally figured out and applied to step 2.
- 03/11: Solved step 4 in about 1 hour of work, wasn’t too bad at all because I originally misread the assignment and had messed around briefly with semaphores during step 2 and removed them.

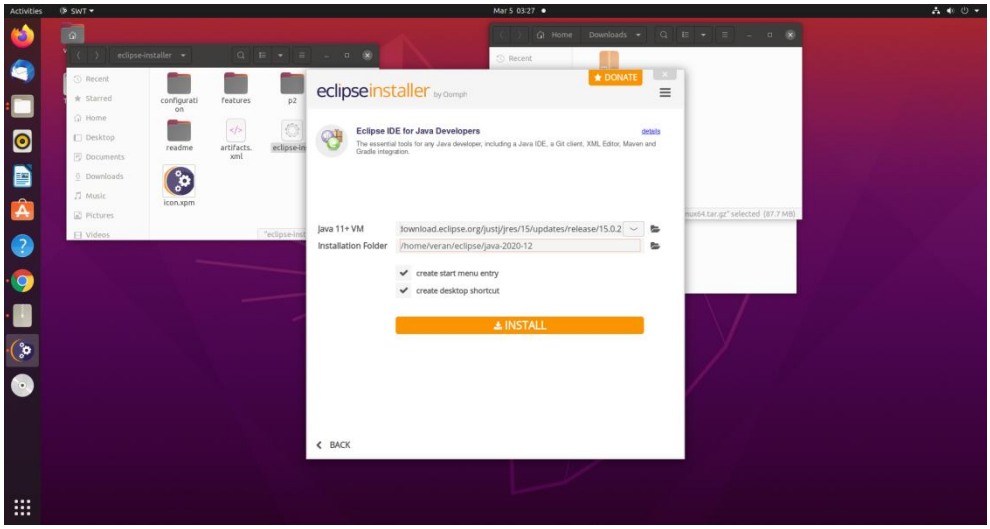
Brian Simpson
OS 4600

Environment Setup:

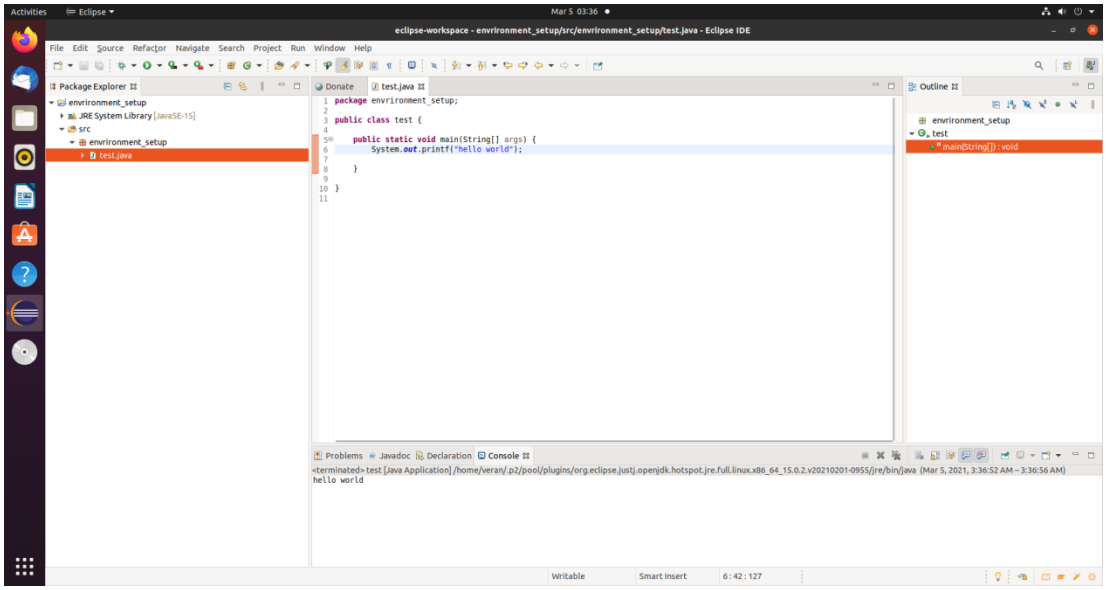
- 

- Downloading Eclipse IDE from their website inside of my Ubuntu VM
- 

Brian Simpson
OS 4600

- 

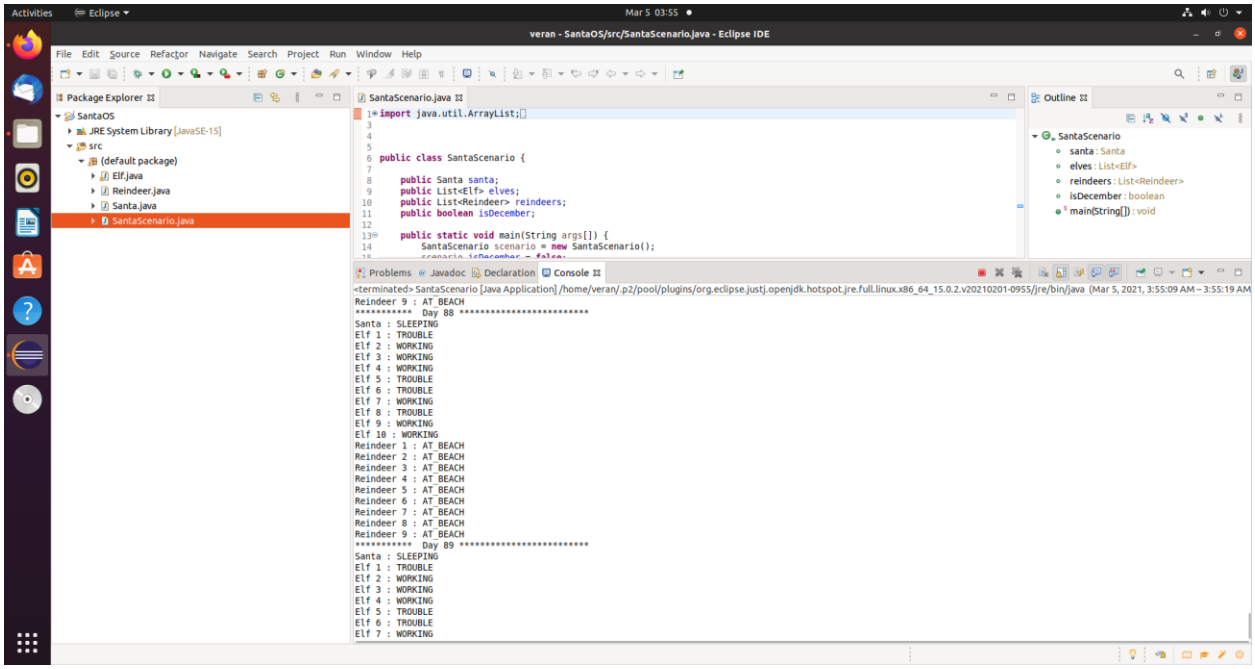
- Installing Eclipse IDE now that it has been downloaded and extracted.

- 

- A basic hello world program from inside of Eclipse IDE, showcasing that it successfully installed and can handle java code.

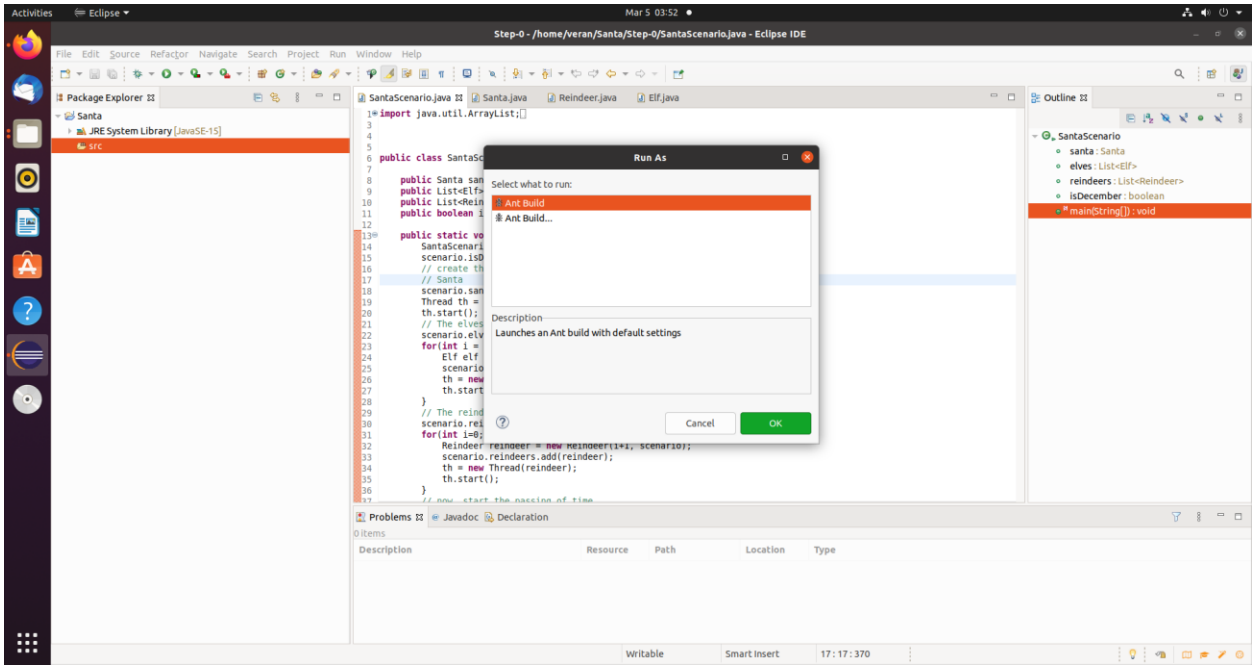
Brian Simpson
OS 4600

Step 0:

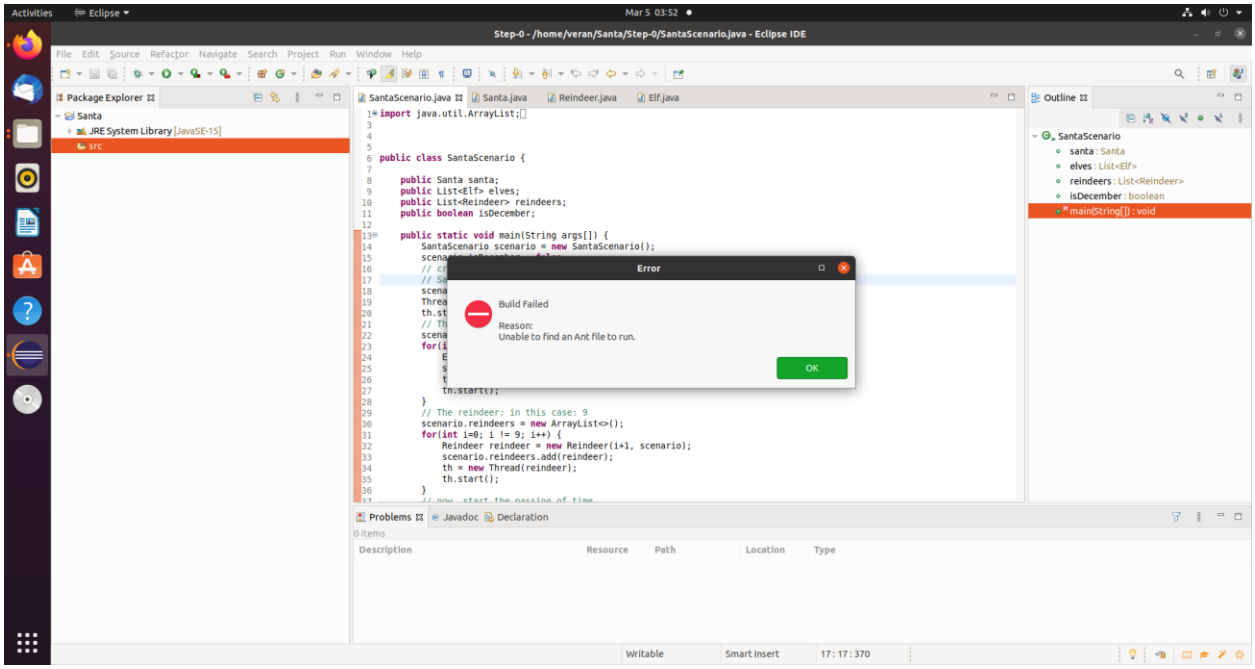
- 

- The initial code from the Santa.zip file running as intended, without any changes having yet been made to the code. Each day the status of the reindeer is given, as well as whether or not each elf is currently troubled or working.

Challenges Faced:

- 

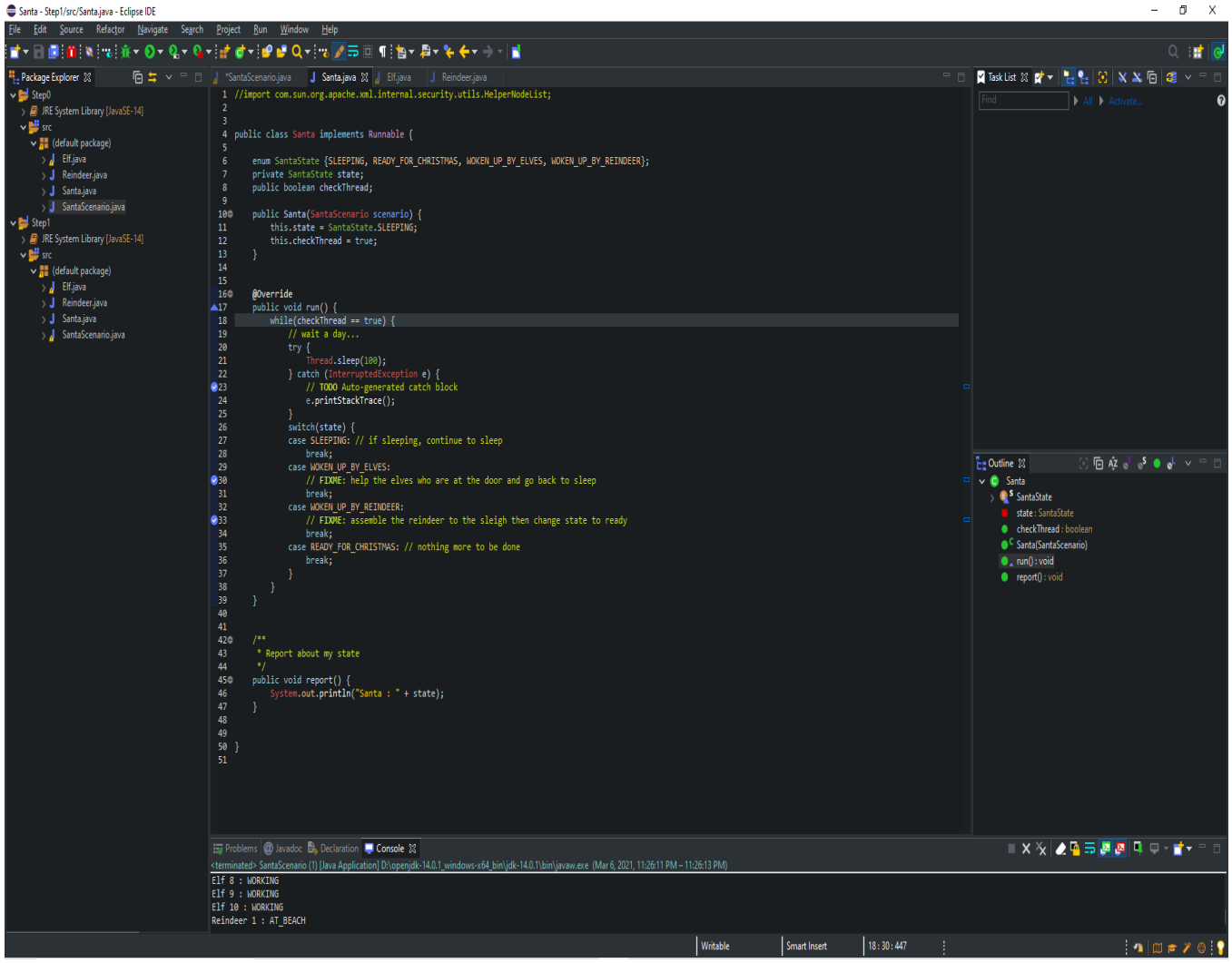
Brian Simpson
OS 4600

- 

- The above two images show a brief error I was encountering when trying to get the code to compile. It was trying to force me to build the program with an ant build instead of running it like a normal program.
- I fixed this by creating a new java project within Eclipse, and importing all of the code from the Santa files present in the old project to the src folder of the new project. This caused whatever error was occurring to go away, and the code compiled normally afterwards.

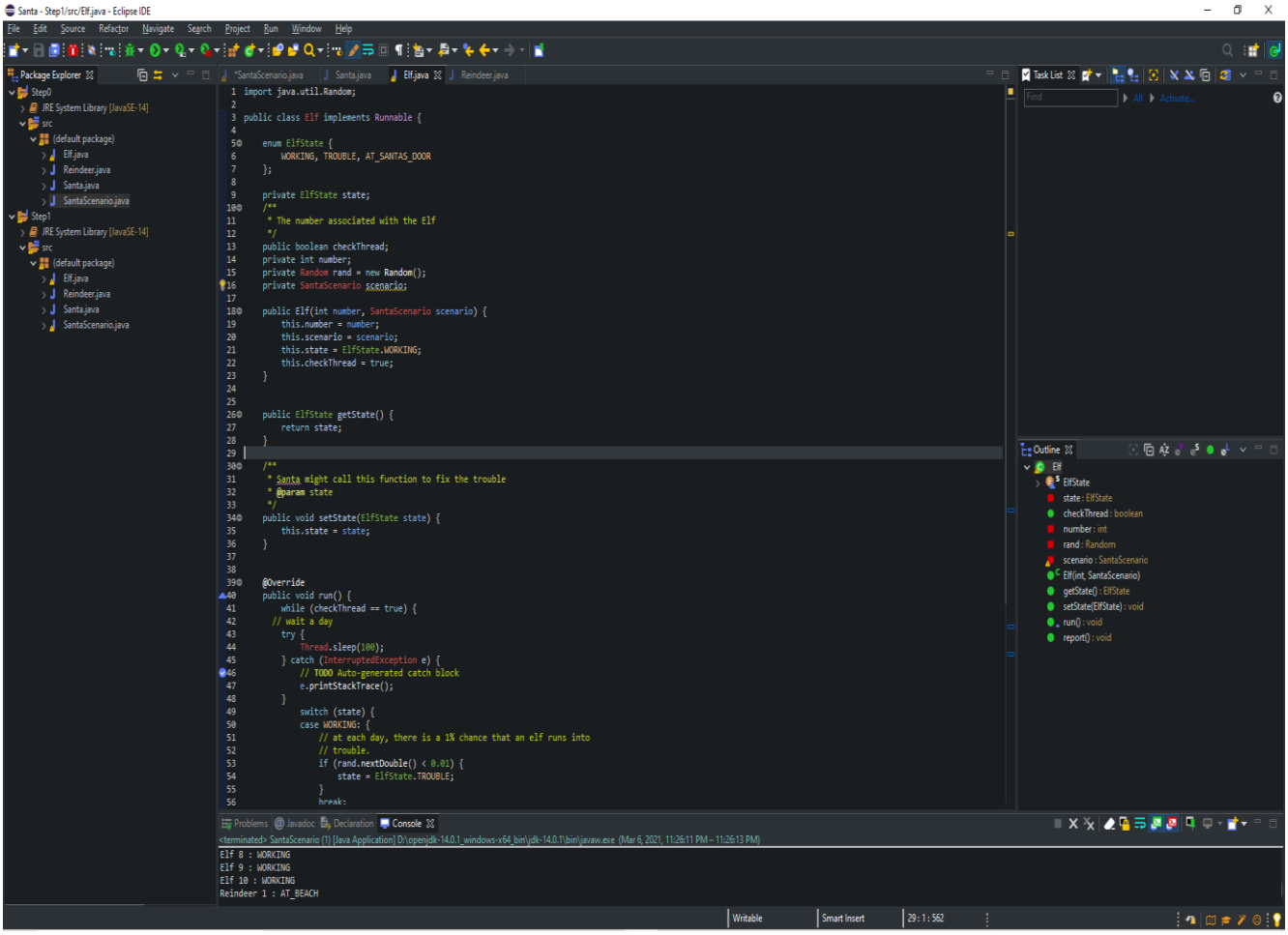
Brian Simpson
OS 4600

Step 1:

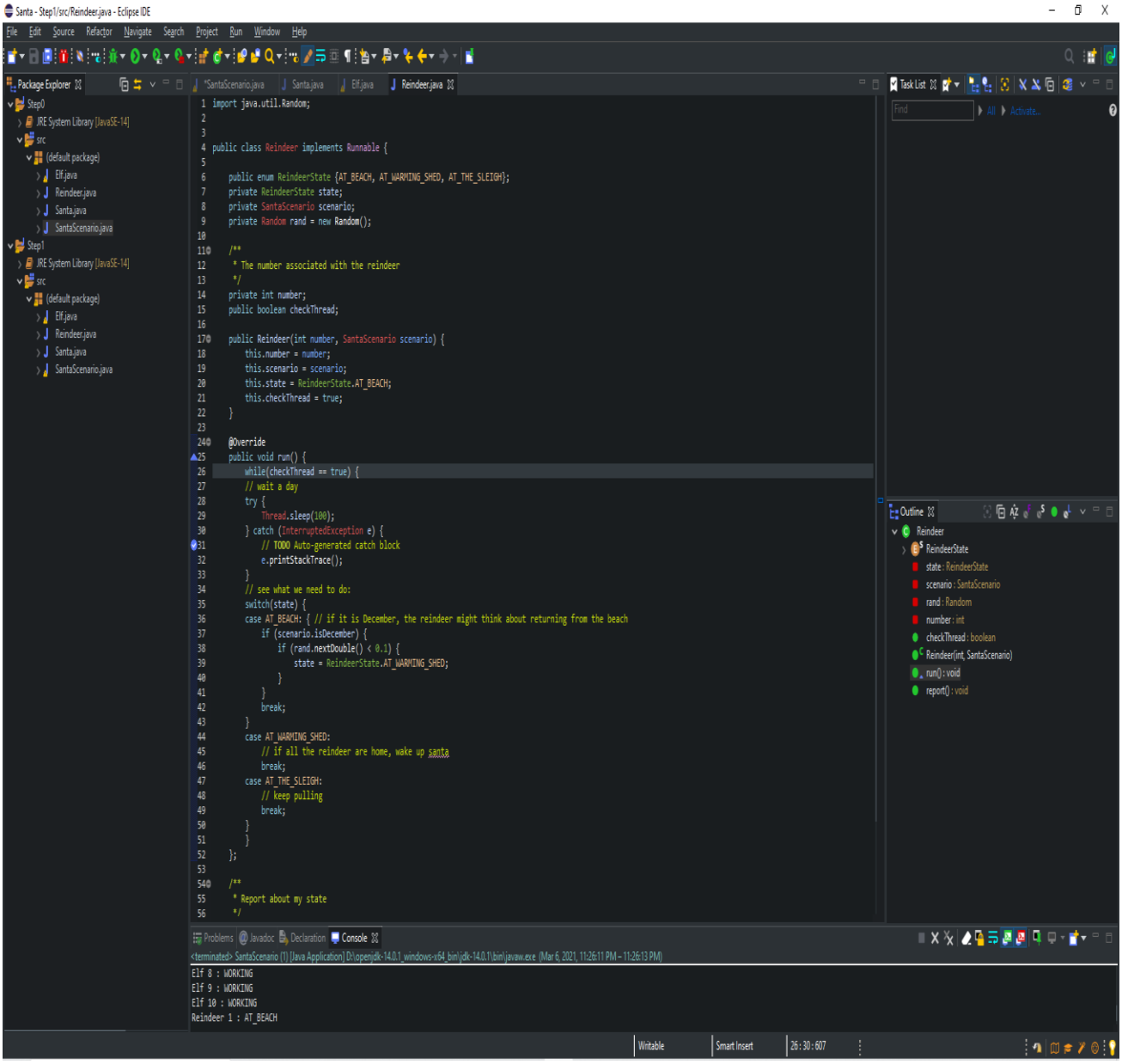
- 

- I added a public Boolean variable to the Santa.java file. Whenever a new instance of Santa is instantiated, I also changed the code in that block to set the value of the new Boolean variable to true. The condition for the while loop in the run method was also changed, now no longer executing whenever the Boolean flag is changed to false, whereas before the loop would execute infinitely.
- Note that I also made the exact same three above changes to Reindeer.java and Elf.java as well, for the exact same reasons. This is shown in the below two screenshots.

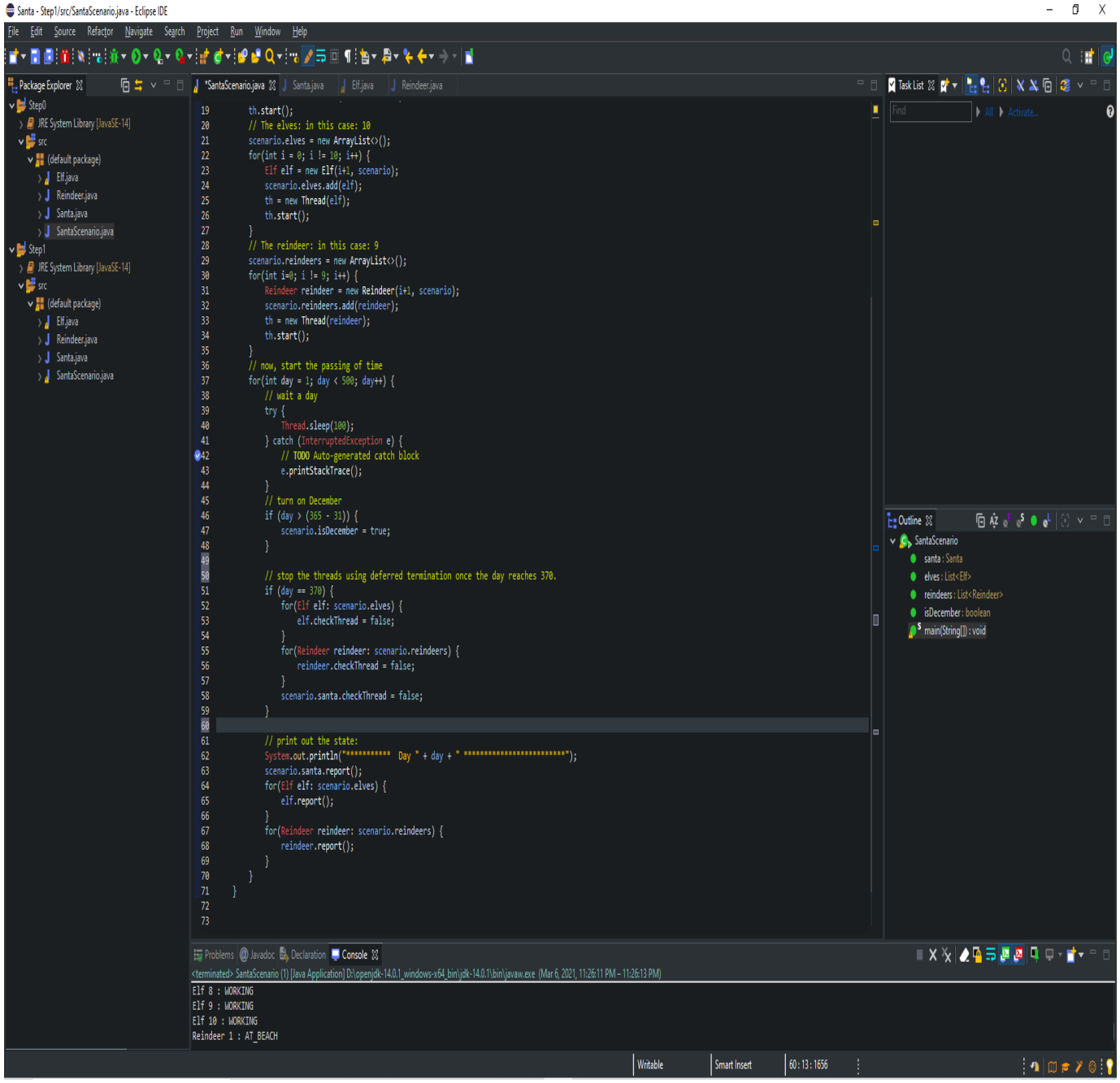
Brian Simpson
OS 4600

- 

Brian Simpson
OS 4600

- 

Brian Simpson
OS 4600

- 

- I added an if statement to the SantaScenario.java file that is only entered once the scenario reaches day 370. When this happens, the newly added boolean flag is changed to false for every reindeer, elf, and Santa. This means that while the threads are not immediately terminated, their status will no longer be checked or updated as the main thread continues to print the days, thus deferred termination as desired.

Brian Simpson
OS 4600

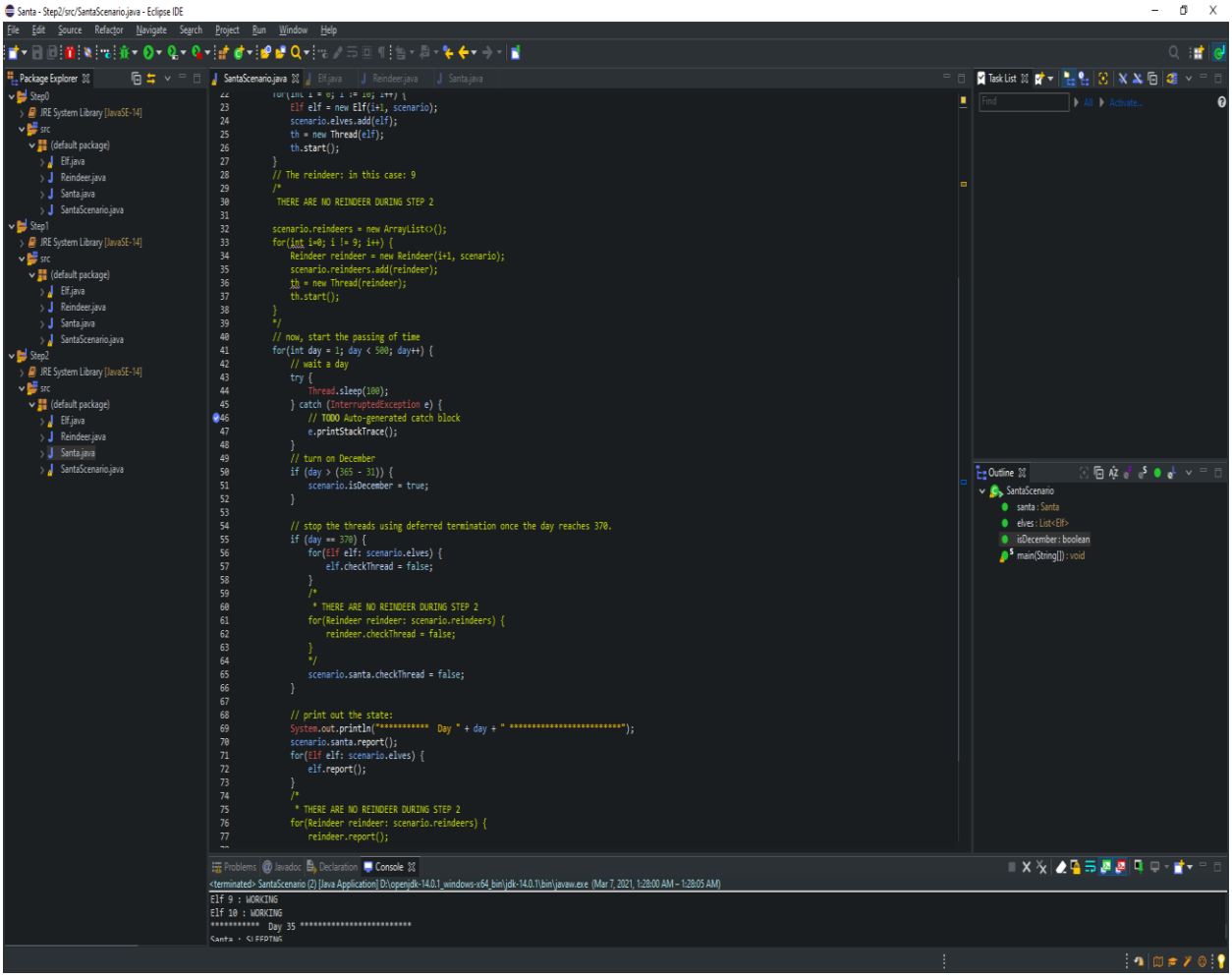
Challenges Faced:

- The first implementation I attempted to use to solve step 1 before I fully understood what deferred termination was, was a dated solution and was not deferring at all.
- I had tried to stop the threads by first putting them to sleep for 5000 milliseconds, and then explicitly stopping them while they were asleep. I was under the impression that killing them while they were in a pause was what was meant by deferring the termination, meaning that they are not killed off until the end of the pause which gives them time to wrap up their tasks
- After I spent some time trying to understand why my solution was not working , I discovered that th.stop is deprecated and no longer supported because it is an unsafe method to call. It was from this point that I began to realize my idea of "deferred termination" was wrong, dug a little deeper, and found some better resources on the topic.
- After I fully grasped what was being asked of me, I then swapped my approach to the one shown above where I simply adjusted the conditions for the while loops for the run methods of the threads.
- Below is an example of what my previously incorrect code for this step looked like at a point.

```
// stop the threads using deferred termination once the day reaches 370.
if (day == 370) {
    Thread.sleep(5000);
    th.stop();
}
```

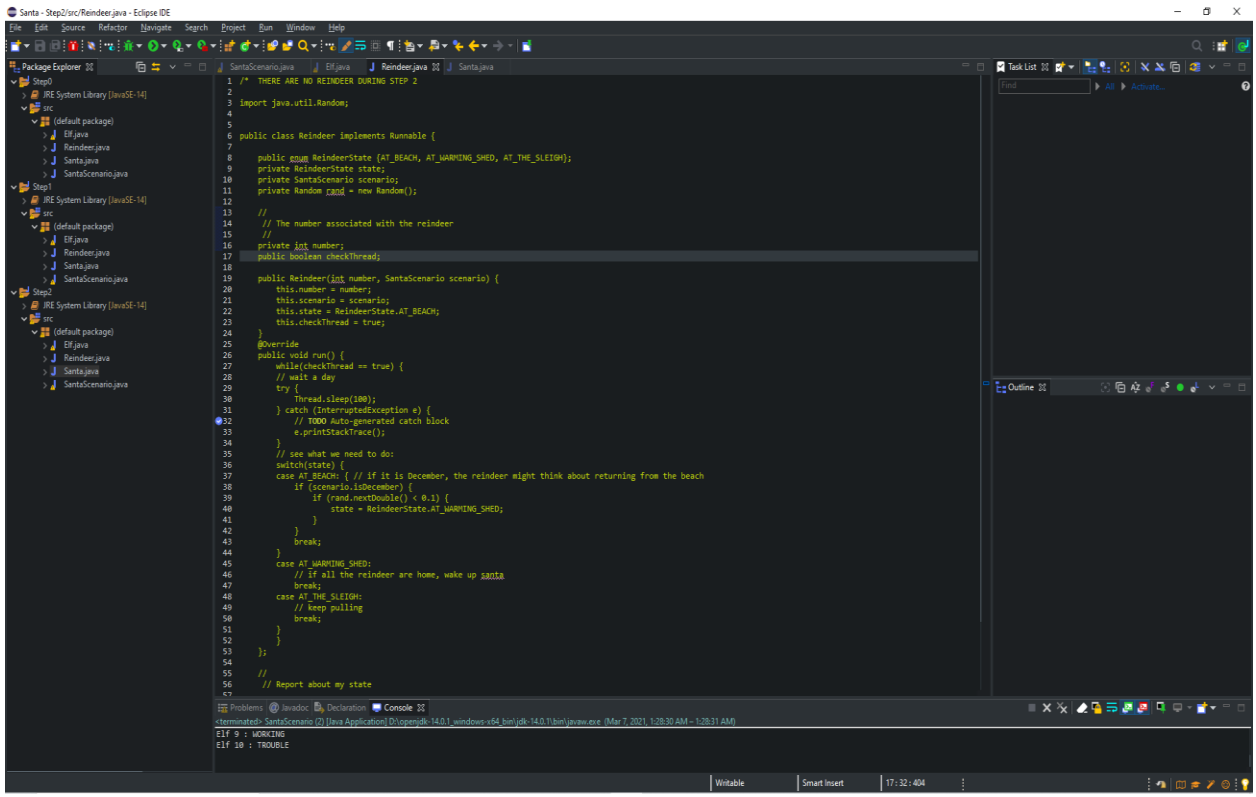
Brian Simpson
OS 4600

Step 2:

- 

- I have commented out all of the code in SantaScenario.java that has anything to do with the reindeer, since step 2 does not involve any reindeer in the simulation per instructions.
- I also commented out the entirety of the Reindeer.java file, since it is never called or used in any way during step 2.

Brian Simpson
OS 4600

- 

- For the next part of step 2, I simply added a single line of code so that whenever an elf reaches the troubled state, it moves to santa's door per instructions for the step.

- ```
  case TROUBLE:
      state = ElfState.AT_SANTAS_DOOR;
      break;
  ```

- For the next step, I added a ListIterator to the sleeping case in Santa.java that iterates through the entire ArrayList of 10 elves every time his sleeping state runs. If even a single elf in that arraylist has their state set to AT_SANTAS_DOOR, then santa's state is set to WOKEN_UP_BY_ELVES as shown below

- ```
  case SLEEPING: // if sleeping, continue to sleep
          ListIterator<Elf> litr = scenario.elves.listIterator();
          while(litr.hasNext()) {
              Elf elf = (Elf) litr.next();
              if(elf.getState().equals(Elf.ElfState.AT_SANTAS_DOOR)) {
                  state = SantaState.WOKEN_UP_BY_ELVES;
              }
          }
      break;
  ```

Brian Simpson
OS 4600

- Finally, when Santa is woken up by elves, he again uses a ListIterator to go through the entire ArrayList of elves. This time he sets the state of any elf who is at his door back to their original working state. Finally, after iterating the entire list, Santa returns to sleep.

```
case WOKEN_UP_BY_ELVES:
    litr = scenario.elves.listIterator();
    while(litr.hasNext()) {
        Elf elf = (Elf) litr.next();
        if(elf.getState().equals(Elf.ElfState.AT_SANTAS_DOOR)) {
            elf.setState(Elf.ElfState.WORKING);
        }
      }
    state= SantaState.SLEEPING;
    break;
```

Challenges Faced:

- This step took me a great deal longer than the first one, as I have not coded anything in Java in quite a long time, and even when I did code in Java it was not very deep beyond the surface level of what was covered in OOP and CS2.
- I struggled to understand how to access the private variables in the other files, even emailing the professor asking if we were allowed to copy and paste the enums from Elf to Santa.
- After a while, I realized that the key was in the scenario variable which held the ArrayList containing all of the elves from the other file, so that I didn't need to move the enum over to the Santa file after all.
- Below is an example of the way I spent a lot of time trying to format my code around at first, before I understood why it was incorrect and that I needed to add the scenario variable to Santa in order to use it to my advantage.
- //for(int i = 0; i != 10; i++) {

  //Elf elf = new Elf(i+1, scenario);

```
//elf.ElfState = elf.getState();

//if(elf.ElfState == Elf.ElfState.AT_SANTAS_DOOR) {

//elf.setState(Elf.ElfState.WORKING);

}

}
```

Step 3:

- Since there are still no reindeer present in step 3, the reindeer.java file and related segments in other files remain commented out identically to how they were shown in step 2.
-

```
case TROUBLE:
    int num_elves_in_trouble = 0;
    boolean elves_already_at_door = false;
    ListIterator<Elf> litr = scenario.elves.listIterator();
    while(litr.hasNext()) {
         Elf elf = (Elf) litr.next();
         if(elf.getState().equals(Elf.ElfState.TROUBLE)) {
             num_elves_in_trouble++;
         }
         if(elf.getState().equals(Elf.ElfState.AT_SANTAS_DOOR)) {
             elves_already_at_door = true;
         }
      }
    if(num_elves_in_trouble >= 3 && elves_already_at_door == false ) {
        litr = scenario.elves.listIterator();
        while(litr.hasNext()) {
            Elf elf = (Elf) litr.next();
            if(elf.getState().equals(Elf.ElfState.TROUBLE)) {
                 elf.state = ElfState.AT_SANTAS_DOOR;
             }
        }
    }
    break;
```

- The above block of code first initiates a list iterator just like step 2, which then cycles through the entire arraylist of elves, also just like step 2. The difference is in what conditions in this arraylist are being checked. For every elf that is in the troubled state, 1 is added to a counter variable. If any elves are detected already at santa's door then a Boolean is set to false so that we do not send any more until they come back. After cycling through the entire list, if we found that there was at least 3 elves in trouble and that there were no elves already at the door, then we again cycle through the entire

arraylist and set every troubled elf to being at the door since the only condition is that they go in groups of 3 or more as clarified in lecture, it does NOT have to be groups of exactly 3.

- The rules for waking santa up and for santa solving the issues of the troubled elves remains the same as step 2, since the elves will not go to the door until they are ready to wake him still, and he will fix all of the elves problems who are at his door when he wakes up still.

Challenges Faced:

- At first I kept noticing that my if statement which checks that at least 3 elves are in trouble and that no elves are currently at the door would execute at the correct time but the elves that were in trouble would not all change to being at santa's door, it would only change one of them. The below screenshot is of my code while experiencing this error.

```
case TROUBLE:
    int num_elves_in_trouble = 0;
    boolean elves_already_at_door = false;
    ListIterator<Elf> litr = scenario.elves.listIterator();
    while(litr.hasNext()) {
         Elf elf = (Elf) litr.next();
         if(elf.getState().equals(Elf.ElfState.TROUBLE)) {
             num_elves_in_trouble++;
         }
         if(elf.getState().equals(Elf.ElfState.AT_SANTAS_DOOR)) {
             elves_already_at_door = true;
         }
     }
    if(num_elves_in_trouble >= 3 && elves_already_at_door == false
        System.out.printf("\n\n%d\n\n", num_elves_in_trouble);
        litr = scenario.elves.listIterator();
        while(litr.hasNext()) {
            Elf elf = (Elf) litr.next();
            if(elf.getState().equals(Elf.ElfState.TROUBLE)) {
                 state = ElfState.AT_SANTAS_DOOR;
             }
        }
    }
    break;
case AT_SANTAS_DOOR:
```

- You may notice that it looks extremely similar to my successful implementation of step 3, and that's because there was only 1 line that was causing the issue.

"state = ElfState.AT_SANTAS_DOOR;" needed to be changed to

"elf.state = ElfState.AT_SANTAS_DOOR;" so that it was referencing the state of each elf as it cycles through the while loop, instead of only changing the state of whichever elf initiated the loop.

Step 4:

- I added this variable to the top of Elf.java

  private static Semaphore sema = new Semaphore(10);

- I then modified the code in the WORKING state of the elf so that whenever the rand chance is successfully rolled, sema.acquire() is called as the elf is moved to the troubled state.

```
// trouble:
if (rand.nextDouble() < 0.01) {
    try {
        sema.acquire();
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    state = ElfState.TROUBLE;

}
break;
```

- Note that despite what the instructions for the assignment say about not being supposed to use interruptedexception, my program would not let me compile without having the acquire call inside of a try catch block like that.

- I released the semaphores whenever they were moved from their troubled state to being at santa's door, per assignment instructions.

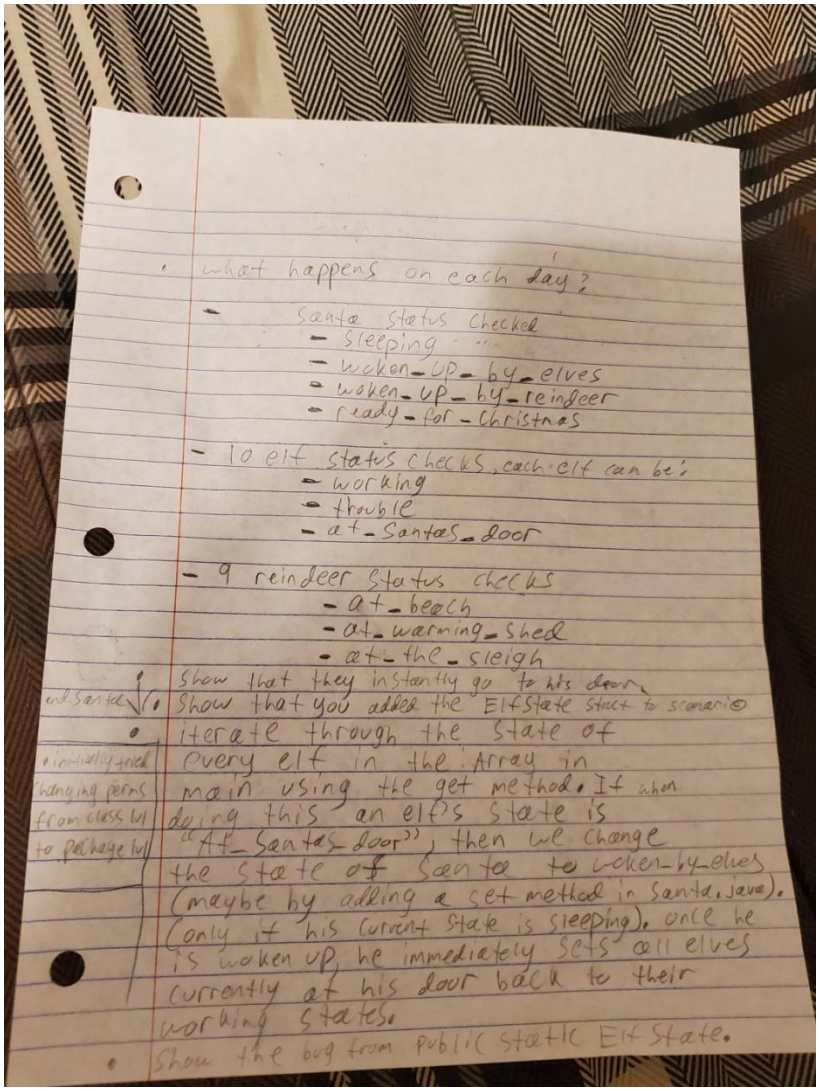
```
}
if(num_elves_in_trouble >= 3 && elves_already_at_door == false ) {
    litr = scenario.elves.listIterator();
    while(litr.hasNext()) {
        Elf elf = (Elf) litr.next();
        if(elf.getState().equals(Elf.ElfState.TROUBLE)) {
            elf.state = ElfState.AT_SANTAS_DOOR;
            sema.release();
        }
    }
}
break;
```

Brian Simpson
OS 4600

Challenges Faced:

- I didn't really have any issues with this portion, it was pretty straightforward and I actually misread the assignment instructions and had put semaphores into my program during step 2 when I was trying to figure it out and removed them to easily implement later for this step.

Paper Notes I Wrote Along The Way:

- Just some scratch notes I made along the way to keep track of things
- 

Brian Simpson
OS 4600